ApexaiQ Technology Asset Management Documentation

Weekly Research Report
Yash Ramesh Fating | Date : 06-mar-2026

1. REST vs GraphQL vs gRPC

Objective

To understand different API communication styles and compare their structure, performance, and use cases.

Introduction

APIs (Application Programming Interfaces) allow different systems to communicate with each other. There are multiple ways to design APIs, among which REST, GraphQL, and gRPC are widely used in modern applications.

REST (Representational State Transfer)

REST is the most commonly used API architecture. It uses HTTP methods such as GET, POST, PUT, and DELETE to perform operations on resources.

- Data is usually returned in JSON format
- Each endpoint represents a resource (e.g., /users, /products)
- Stateless communication

Example:
GET /users → Fetch all users

Advantages:

- Simple and widely supported
- Easy to understand and implement

Limitations:

- Over-fetching or under-fetching of data
- Multiple requests may be needed for related data

GraphQL

GraphQL is a query-based API approach where the client specifies exactly what data it needs.

- Single endpoint for all requests
- Client controls the response structure
- Reduces unnecessary data transfer

Example Query:

```
{
  user(id: 1) {
    name
    email
  }
}
```

Advantages:

- Efficient data fetching
- Flexible for frontend applications

Limitations:

- More complex to implement
- Requires proper query validation

---

gRPC (Google Remote Procedure Call)

gRPC is a high-performance API framework that uses Protocol Buffers (binary format) instead of JSON.

- Faster than REST and GraphQL
- Supports streaming

- Strongly typed contracts

Advantages:

- High performance and low latency
- Ideal for microservices communication

Limitations:

- Not browser-friendly
- Harder to debug compared to REST

Comparison Summary

Feature	REST	GraphQL	gRPC
Data Format	JSON	JSON	Protobuf
Performance	Medium	Medium	High
Flexibility	Low	High	Medium
Complexity	Low	Medium	High
Use Case	Web APIs	Frontend apps	Microservices

Conclusion

REST is best for simple APIs, GraphQL is suitable for flexible data fetching, and gRPC is ideal for high-performance backend systems.

---

2. API Authentication Methods

Objective

To understand different methods used to secure APIs.

Introduction

API authentication ensures that only authorized users or systems can access the API.

Common Methods

1. API Key

A unique key is passed in the request header.

Example:

x-api-key: abc123

- Simple but less secure

---

2. Basic Authentication

Uses username and password encoded in Base64.

- Easy to implement
- Not recommended for production without HTTPS

---

3. Bearer Token (JWT)

Uses tokens for authentication.

Example:

Authorization: Bearer <token>

- Secure and widely used
- Token contains user information

---

4. OAuth 2.0

Used for third-party access (e.g., login with Google).

- Secure and scalable
- Supports access delegation

Conclusion

Bearer tokens and OAuth are preferred for secure applications, while API keys are suitable for basic use cases.

---

3. API Rate-Limiting Algorithms

Objective

To control the number of API requests and prevent abuse.

Introduction

Rate limiting restricts how many requests a user can make in a given time.

Algorithms

1. Fixed Window

- Requests allowed per fixed time interval
- Simple but can cause burst traffic

---

2. Sliding Window

- More flexible than fixed window
- Tracks requests over a rolling time period

---

3. Token Bucket

- Tokens are added at a fixed rate
- Requests consume tokens
- Allows bursts while controlling overall rate

---

4. Leaky Bucket

- Requests are processed at a constant rate
- Excess requests are dropped

Conclusion

Token bucket is widely used because it balances flexibility and control.

---

4. API Versioning Strategies

Objective

To manage changes in APIs without breaking existing clients.

Introduction

As APIs evolve, versioning ensures backward compatibility.

Strategies

1. URI Versioning

/api/v1/users

- Simple and commonly used

---

2. Header Versioning

Accept: application/vnd.api.v1+json

- Keeps URLs clean

---

3. Query Parameter

/users?version=1

- Easy but less preferred

---

4. Content Negotiation

- Version specified in headers
- More flexible but complex

Conclusion

URI versioning is most widely used due to its simplicity and clarity.

---

5. API Security Best Practices

Objective

To ensure APIs are protected against attacks and misuse.

Key Practices

1. Use HTTPS

- Encrypts data during transmission

---

2. Authentication and Authorization

- Verify identity and permissions

---

3. Input Validation

- Prevents injection attacks

---

4. Rate Limiting

- Protects against abuse and DDoS

---

5. Logging and Monitoring

- Helps detect suspicious activity

---

6. Use Secure Tokens

- Prefer JWT or OAuth

---

7. Avoid Exposing Sensitive Data

- Never return passwords or secrets